



ZeRO-Offload: Democratizing Billion-Scale Model Training

Jie Ren, *UC Merced*; Samyam Rajbhandari, Reza Yazdani Aminabadi, and Olatunji Ruwase, *Microsoft*; Shuangyan Yang, *UC Merced*; Minjia Zhang, *Microsoft*; Dong Li, *UC Merced*; Yuxiong He, *Microsoft*

<https://www.usenix.org/conference/atc21/presentation/ren-jie>

**This paper is included in the Proceedings of the
2021 USENIX Annual Technical Conference.**

July 14–16, 2021

978-1-939133-23-6

**Open access to the Proceedings of the
2021 USENIX Annual Technical Conference
is sponsored by USENIX.**

ZeRO-Offload: Democratizing Billion-Scale Model Training

Jie Ren
UC Merced

Samyam Rajbhandari
Microsoft

Reza Yazdani Aminabadi
Microsoft

Olatunji Ruwase
Microsoft

Shuangyan Yang
UC Merced

Minjia Zhang
Microsoft

Dong Li
UC Merced

Yuxiong He
Microsoft

Abstract

Large-scale model training has been a playing ground for a limited few users, because it often requires complex model refactoring and access to prohibitively expensive GPU clusters. ZeRO-Offload changes the large model training landscape by making large model training accessible to nearly everyone. It can train models with over 13 billion parameters on a single GPU, a 10x increase in size compared to popular framework such as PyTorch, and it does so without requiring any model change from data scientists or sacrificing computational efficiency.

ZeRO-Offload enables large model training by offloading data and compute to CPU. To preserve compute efficiency, it is designed to minimize data movement to/from GPU, and reduce CPU compute time while maximizing memory savings on GPU. As a result, ZeRO-Offload can achieve 40 TFlop-s/GPU on a single NVIDIA V100 GPU for 10B parameter model, compared to 30TF using PyTorch alone for a 1.4B parameter model, the largest that can be trained without running out of memory on GPU. ZeRO-Offload is also designed to scale on multiple-GPUs when available, offering near-linear speedup on up to 128 GPUs. Additionally, it can work together with model parallelism to train models with over 70 billion parameters on a single DGX-2 box, a 4.5x increase in model size compared to using model parallelism alone.

By combining compute and memory efficiency with ease-of-use, ZeRO-Offload democratizes large-scale model training making it accessible to even data scientists with access to just a single GPU.

1 Introduction

Since the advent of the attention-based deep learning (DL) models in 2017, we have seen an exponential growth in DL model size, fueled by substantial quality gains that these attention based models can offer with the increase in the number of parameters. For example, the largest language model in literature had less than 100M parameters in 2017. It grew to over 300M with BERT [6] in 2018, and increased to tens of billions in 2019 with models such as GPT-2 [3], T5 [20], Megatron-LM [28] and Turing-NLG [25]. Today, the largest

language model GPT-3 [2] has a staggering number of 175B parameters. With the three orders of magnitude growth in model size since 2017, the model accuracy continues to improve with the model size [12]. Recent studies in fact show that larger models are more resource-efficient to train than smaller ones [12] for a given accuracy target. As a result, we expect the model size to continue growing in the future.

However, accessibility to large model training is severely limited by the nature of state-of-art system technologies. Those technologies make entry into the large model training space prohibitively expensive. To be more specific, distributed parallel DL training technologies such as pipeline parallelism [10], model parallelism [28], and ZeRO [21] (Zero Redundancy Optimizer) allow transcending the memory boundaries of single GPU/accelerator device by splitting the model states (parameters, gradients and optimizer states) across multiple GPU devices, enabling massive models that would otherwise simply not fit in a single GPU memory. All record-breaking large models such as GPT-2, Megatron-LM, Turing-NLG, and GPT-3, were trained using a combination of the aforementioned technologies. However, all of these DL parallel technologies require having enough GPU devices such that the aggregated GPU memory can hold the model states required for the training. For example, training a 10B parameter model efficiently requires a DGX-2 equivalent node with 16 NVIDIA V100 cards, which costs over 100K, beyond the reach of many data scientists, and even many academic and industrial institutions.

Heterogeneous DL training is a promising approach to reduce GPU memory requirement by exploiting CPU memory. Many efforts have been made in this direction [8, 9, 11, 17, 23, 23, 24, 32–34]. Nearly all of them target CNN based models, where activation memory is the memory bottleneck, and model size is fairly small (less than 500M). However, the primary memory bottleneck for recent attention based large model training are the model states, instead of activation memory. There is an absence in literature studying these workloads for heterogeneous DL training. Additionally, existing efforts on heterogeneous training are further limited in two major ways: i) nearly all of them exploit CPU memory, but not CPU compute, which we show can be used to significantly reduce the CPU-GPU communication overhead, and

ii) they are mostly designed for and evaluated on single GPU [9, 11, 23, 34], without a clear path to scaling efficiently on multiple GPUs, which is crucial for large model training.

Addressing the aforementioned limitation, we attempt to democratize large model training by developing ZeRO-Offload, a novel heterogeneous DL training technology designed specifically for large model training. ZeRO-Offload exploits both CPU memory and compute for offloading, while offering a clear path towards efficiently scaling on multiple GPUs by working with ZeRO-powered data parallelism [21]. Additionally, our first principle analysis shows that ZeRO-Offload provides an optimal and the only optimal solution in maximizing memory saving while minimizing communication overhead and CPU compute overhead for large model training.

ZeRO-Offload is designed around three main pillars: i) Efficiency, ii) Scalability, and iii) Usability.

Efficiency: The offload strategy in ZeRO-Offload is designed with the goal of achieving comparable compute efficiency to the state-of-art non-offload strategies but for significantly larger models. To achieve this goal, we rely on first principle analysis to identify a *unique optimal* computation and data partitioning strategy between CPU and GPU devices. This strategy is optimal in three key aspects: i) it requires orders-of-magnitude fewer computation on CPU compared to GPU, preventing the CPU compute from becoming a performance bottleneck, ii) it minimizes the communication volume between CPU and GPU preventing communication from becoming a bottleneck, and iii) it provably maximizes memory savings on GPU while achieving minimum communication volume.

Our analysis shows that to be optimal in the aforementioned regards, we must offload the gradients, optimizer states and optimizer computation to CPU, while keeping the parameters and forward and backward computation on GPU. This strategy enables a 10x increase in model size, with minimum communication and limited CPU computation, which allows us to train 13B parameters on a single NVIDIA V100 GPU at 40 TFLOPS, compared to 30 TFLOPS on the same GPU with 1.2B parameters, the largest model that can be trained without any CPU offloading.

Offloading optimizer computation requires CPU to perform $O(M)$ computation compared to $O(MB)$ on GPU where M and B are the model size and batch sizes respectively. In most cases, the batch size is large, and CPU computation is not a bottleneck, but for small batch sizes, the CPU compute can be a bottleneck. We address this using two optimizations: i) an efficient *CPU optimizer* that is up to 6x faster than the-state-of-art, and ii) One-step *delayed parameter update* that allows overlapping the CPU optimizer step with GPU compute, while preserving accuracy. Together, they preserve efficiency for ZeRO-Offload even with small batch sizes.

Scalability: Good scalability is crucial to take advantage of multiple GPUs that may be available to some data scientists. In the DL community, data parallelism is generally

<pre>import torch model = BuildModel(config) optimizer = Optimizer(model) for batch in batches: loss = model(batch) loss.backward() optimizer.step()</pre>	<pre>import torch import deepspeed model = BuildModel(config) optimizer = Optimizer(model) model = deepspeed.initialize(model, optimizer) for batch in batches: loss = model(batch) model.backward(loss) model.step()</pre>
--	--

Figure 1: ZeRO-Offload can be enabled with a few lines of change. The code on left shows a standard training pipeline, while the right shows the same pipeline with ZeRO-Offload enabled.

used as the de facto standard to scale DL training to multiple GPUs [5, 26, 35]. However, it is not designed to work with heterogeneous training and presents scalability challenges because of the replication of data and computation in data parallel training. Data parallel training replicates all the model states such as optimizer states, parameters, and gradients, and it also replicates the optimizer computation on each GPU. Therefore, offloading model states or optimizer computation to CPU in combination with data parallelism will result in significant replication of communication and CPU compute: increase the CPU memory requirement proportionally to the data parallelism degree while limiting throughput scalability due to the increased communication.

To address these challenges, ZeRO-Offload combines unique optimal offload strategy with ZeRO [21] powered data parallelism instead of traditional data parallelism. The symbiosis allows ZeRO-Offload to maintain a single copy of the optimizer states on the CPU memory regardless of the data parallel degree. Furthermore, it keeps the aggregate communication volume between GPU and CPU, as well as the aggregate CPU computation a constant regardless of data parallelism, allowing ZeRO-Offload to effectively utilize the linear increase in CPU compute with the increase in the data parallelism degree. As a result, ZeRO-Offload achieves excellent scalability on up to 128 GPUs.

In addition to working with ZeRO powered data parallelism, ZeRO-Offload can be combined with model parallelism [27, 28] to achieve higher memory savings, when multiple GPUs are available.

Usability: ZeRO-Offload is available as part of an Open-Source PyTorch library, DeepSpeed (www.deepspeed.ai). Unlike most strategies discussed in Section 2, ZeRO-Offload does not require model refactoring to work. In fact, PyTorch users can enable ZeRO-Offload with few lines of code change to their existing training pipeline as shown in Figure 1, allowing to train 10x larger models easily.

Contributions. To the best of our knowledge, ZeRO-Offload is the first fully distributed all-reduced based training framework using CPU memory and computation resources to train large-scale models. We summarize contributions as follows:

- A unique optimal offload strategy for heterogeneous large model training on GPU + CPU system that enables 10x larger model on a single GPU without sacrificing efficiency (Sec. 3 and Sec. 4.1).
- Highly scalable multi-GPU design through i) a symbiotic combination of offload strategy with ZeRO powered data parallelism (Sec. 4.2), allowing ZeRO-Offload to achieve near-linear scalability, and ii) seamless integration with model-parallel training [28], enabling even larger models than using ZeRO-Offload or model parallelism alone (Sec. 4.2).
- Open-source implementation of ZeRO-Offload in PyTorch.
- Extensive evaluation demonstrating i) *Model Scale*: 10x increase in model size with up to 13B on a single GPU and 4x increase in model size over model parallelism with up to 70B parameters on a DGX-2 node. ii) *Efficiency*: Over 40 TFlops for a 10B parameters on a single NVIDIA V100, compared to 30 TFLOPS on the same GPU with 1.4B parameters, the largest model that can be trained without any CPU offloading; Outperform two state-of-the-art heterogeneous DL training frameworks by 22% and 37% respectively on a single GPU. iii) *Scalability*: Near-perfect linear scalability for a 10B parameter model on up to 128 GPUs. iv) CPU overhead reduction with our ADAM implementation with 6x speedup over PyTorch optimizer and up to 1.5X improvement in end-to-end throughput with delayed parameter update optimizations (Sec. 6).

2 Background and Related Work

Memory consumption in large model training. The full spectrum of memory consumption during DL model training can be classified into two parts: i) model states and ii) residual states [21]. Model states include parameters, gradients, and optimizer states (such as momentum and variances in Adam [13]); Residual states include activations, temporary buffers, and unusable fragmented memory.

Model states are the primary source of memory bottleneck in large model training. We consider the memory consumption due to model states for large transformer models such as Megatron-LM (8 billion) [28], T5 (11 billion) [20], and Turing-NLG [25] (17.2 billion). They are trained with float-16 mixed precision training [16] and Adam optimizer [13].

Mixed precision training often keeps two copies of the parameters, one in float-16 (fp16) and the other in float-32 (fp32). The gradients are stored in fp16. In addition to the parameters and gradients, the Adam optimizer keeps track of the momentum and variance of the gradients. These optimizer states are stored in fp32. Therefore, training a model in mixed precision with the Adam optimizer requires at least 2 bytes of memory for each fp16 parameter and gradient, and 4 byte of memory for each fp32 parameter, and the momentum and variance of each gradient. In total, a model with M parameters requires $16 \times M$ bytes of memory. Therefore, the model states

for Megatron-LM, T5 and Turing-NLG require 128 GB, 176 GB and 284 GB, respectively, which are clearly beyond the memory capacity of even the current flagship NVIDIA A100 GPU with 80 GB of memory.

Significant amount of work has been done in the recent years to enable large model training, which requires more memory than what is available on a single GPU to fit these model and residual states. These efforts can be classified broadly into two categories: i) scale-out training and ii) scale-up training based approaches. We discuss them as follows.

Scale out large model training. Scale-out training uses aggregate memory of multiple GPUs to satisfy the memory requirement for large model training. Two prominent examples of scale out training is model parallelism [5, 28] and pipeline parallelism [7, 10], both partitioning the model states and the residual states across multiple GPUs. Model parallelism [5, 28] partitions the model vertically and distributes the model partitions to multiple GPU devices in order to train large models. Pipeline parallelism [7, 10] on the other hand parallelizes model training by partitioning the model horizontally across layers. Both of these approaches must change the user model to work, therefore can limit usability.

A recent work, ZeRO [21], provides an alternative to model and pipeline parallelisms to train large models. ZeRO splits the training batch across multiple GPUs similar to data parallel training [5, 26, 35], but unlike data parallel training which replicates all the model states on each GPU, ZeRO partitions them across all GPUs, and uses communication collectives to gather individual parameters as needed during the training. ZeRO does not require changes to the user model to work, making it more generic than model or pipeline parallel training. It also offers better compute efficiency and scalability.

Despite the ability of model and pipeline parallelisms, and ZeRO to train large models, they all require multiple GPUs such that the aggregate GPU memory can hold the model and residual states for training large models. In contrast, ZeRO-Offload is designed to fit a larger model by offloading model states to CPU memory and can train a 10x larger model on a single GPU without sacrificing efficiency. When multiple GPUs are available, ZeRO-Offload is designed to work together with ZeRO to offer excellent scalability, or in conjunction with model parallelism to fit even larger model sizes that is not possible with ZeRO-Offload or model parallelism alone.

Scale up large model training. Existing work scales up model size in a single GPU through three major approaches. The first approach trades computation for memory saving from activations (residual memory) by recomputing from checkpoints [4]. The second approach uses compression techniques such as using low or mixed precision [16] for model training, saving on both model states and activations. The third approach uses an external memory such as the CPU memory as an extension of GPU memory to increase memory capacity during training [8, 9, 11, 17, 23, 24, 33].

Our work, ZeRO-Offload falls under the third approach.

Unlike ZeRO-Offload, the above efforts only offload data to CPU but not compute, and they use smaller models training. Furthermore, none of the above works is communication optimal, leading to extra communication between CPU and GPU and hurting training throughput. In contrast, a recent work called L2L [18] can enable multi-billion parameter training by managing memory usage in GPU layer by layer. In particular, L2L synchronously moves tensors needed in the upcoming layer into GPU memory for computation and keeps the rest of tensors into CPU memory for memory saving. In comparison to ZeRO-Offload, it offers limited efficiency due to extra communication overhead, does not offer a way to scale out across devices, and requires model refactoring, making it difficult to use.

ZeRO powered data parallel training. ZeRO-Offload works with ZeRO to scale DL training to multiple GPUs. ZeRO has three stages, ZeRO-1, ZeRO-2 and ZeRO-3 corresponding to the partitioning of the three different model states, optimizer states, gradients and parameters, respectively. ZeRO-1 partitions the optimizer states only, while ZeRO-2 partitions gradients in addition to optimizer states, and ZeRO-3 partitions all model states. ZeRO-Offload works symbiotically with ZeRO-2, and therefore we discuss it further.

In ZeRO-2, each GPU stores a replica of all the parameters, but only updates a mutually exclusive portion of it during the parameter update at the end of each training step. As each GPU only updates a portion of the parameters, they only store optimizer states and gradients required to make that update. After the update, each GPU sends its portion of the updated parameters to all the other GPUs using an `all-gather` communication collective. ZeRO-2 computation and communication schedule is described below:

During the forward pass, each GPU computes the loss with respect to a different mini-batch. During the backward propagation, as each gradient is computed, it is averaged using a `reduce` operator at the GPU/GPUs that owns the gradient or part of the gradient. After the backward pass, each GPU updates its portion of the parameters and optimizer states using the averaged gradients corresponding to that portion. After this, an `all-gather` is performed to receive the rest of the parameter update computed on other GPUs.

3 Unique Optimal Offload Strategy

ZeRO-Offload is designed to enable efficient large model training on a single or multiple GPUs by offloading some of the model states from GPU to CPU memory during training. As discussed in Sec. 2, model states: parameters, gradients, and the optimizer states, are the primary source of memory bottleneck in large model training. By offloading some of these model states to CPU, ZeRO-Offload can enable training of significantly larger models¹. However, identifying the

¹ZeRO-Offload only offloads model states. Offloading secondary sources of memory bottleneck such as activation memory is beyond scope of our

optimal offloading strategy is non-trivial. There are numerous ways to offload model states to CPU memory, each with a different trade-off in terms of CPU computation, and GPU-CPU communication, both of which can limit the training efficiency.

To identify the optimal offload strategy, ZeRO-Offload models the DL training as data-flow graph and uses first principle analysis to efficiently partition this graph between CPU and GPU devices. ZeRO-Offload partitions the graph in a way that is *optimal* in three key aspects: i) it requires orders-of-magnitude fewer computation on CPU compared to GPU, which prevents CPU from becoming a performance bottleneck (Sec. 3.1), ii) it guarantees the minimization of communication volume between CPU and GPU memory (Sec. 3.3), and iii) it provably maximizes the memory savings while achieving minimum communication volume (Sec. 3.4). In fact, ZeRO-Offload can achieve high efficiency during training that is comparable to non-offload training and it is *unique optimal*, meaning no other solution can offer better memory savings without increasing the communication volume or increasing CPU computation.

In this section, we discuss the derivation of our unique optimal offload strategy. Our strategy is specifically designed for *mixed precision training with Adam optimizer* which is the de facto training recipe for large model training.

3.1 DL Training as a Data-Flow Graph

The DL training workload can be represented as a weighted directed graph of data and computation, as shown in Figure 2, where the circular nodes represents model states (parameter16, gradient16, parameter32, momentum32, variance32), and the rectangular nodes represents computation (forward, backward, param update). The edges in the graph represents the data flow between the nodes, and the weight of an edge is the total data volume in bytes that flows through it during any given training iteration. For a model with M parameters, the weight of the edges in this graph is either $2M$ where the source node produces fp16 model states, or $4M$ where the source node produces fp32 model states.

An offload strategy between GPU and CPU can be represented using a two-way partitioning of this graph, such that computation nodes in a partition would be executed on the device that owns the partition, and the data nodes in the partition will be stored on device that owns the partition. The total data volume that must be communicated between GPU and CPU is given by the weight of edges running across two partitions.

There are numerous ways to partition this graph. In the following sections, we use first principles to simplify the data

offload strategy. Given that they are significantly smaller than model states, we ignore them for the purpose of our analysis. Furthermore, the first and second approaches described in Sec. 2 can be used in conjunction with ZeRO-Offload to reduce activation memory

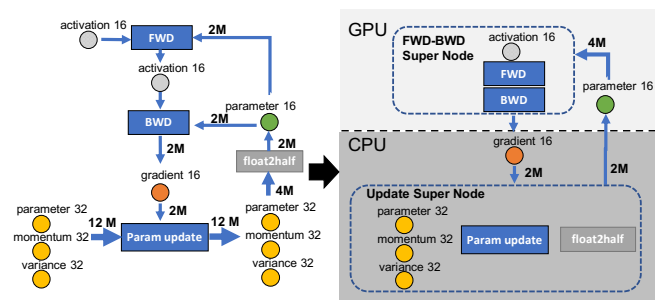


Figure 2: The dataflow of fully connected neural networks with M parameters. We use activation checkpoint to reduce activation memory to avoid activation migration between CPU and GPU.

flow graph to reduce the number of possible choices based on three different efficiency metric: i) CPU computation overhead, ii) communication overhead, and iii) memory savings.

3.2 Limiting CPU Computation

The CPU computation throughput is multiple orders of magnitude slower than the GPU computation throughput. Therefore, offloading large computation graph to CPU will severely limit training efficiency. As such, we must avoid offloading compute intensive components to the CPU.

The compute complexity of DL training per iteration is generally given by $O(MB)$, where M is the model size and B is the effective batch size. To avoid CPU computation from becoming a bottleneck, only those computations that have a compute complexity lower than $O(MB)$ should be offloaded to CPU. This means that the forward propagation and backward propagation both of which have a compute complexity of $O(MB)$ must be done on GPU, while remaining computations such as norm calculations, weight updates etc that have a complexity of $O(M)$ may be offloaded to CPU.

Based on this simple observation we fuse the forward and backward nodes in our data flow graph into a single super-node (FWD-BWD) and assign it to GPU.

3.3 Minimizing Communication Volume

The CPU memory bandwidth is at least an order of magnitude faster than the PCI-E bandwidth between CPU and GPU, while the GPU memory is another order of magnitude faster than even the CPU memory. Therefore, we must minimize the communication volume between CPU and GPU memory to prevent the PCI-E bandwidth from becoming a training performance bottleneck. To do so we must first identify the theoretical minimum communication volume for a model-state offload strategy.

The minimum communication volume for any model-state offload strategy is given by $4M$ ². Note that after fusing the

²Please note that it is possible to reduce the communication volume further by only offloading partial model states. For simplification, we assume

forward and backward into a single super-node as discussed in Sec. 3.2, each node in our data flow graph is part of a cycle. Therefore, any partitioning of this graph would require cutting at least two edges, each of which has a edge weight of at least $2M$, resulting in a total communication of at least $4M$.

If we choose to limit the communication volume to this bare minimum, we can greatly simplify our data-flow graph and reduce the number of partitioning strategies to a handful:

Creating fp32 super-node. Notice that any partitioning strategy that does not co-locate the fp32 model states with their producer and consumer nodes cannot achieve the minimum communication volume of $4M$. Such a partition must cut at least one edge with a weight of $4M$, and the other with at least $2M$, resulting in a communication volume of at least $6M$. Therefore, to achieve the minimum communication volume, all offload strategies must co-locate fp32 model states with their producer and consumer operators, i.e., the fp32 model states (momentum32, variance32 and parameter32) must be co-located with the *Param Update* and the *float2half* computation.

This constraint allows us to treat all the aforementioned fp32 data and compute nodes in the data flow graph as a single super-node that we refer to as *Update Super*. We show this reduced data flow graph in Figure 2, consisting of only four nodes: *FWD-BWD Super* node, *p16* data node, *g16* data node, and *Update Super* node.

p16 assignment. To achieve the minimum communication volume, *p16* must be co-located with *FWD-BWD Super* because the edge weight between these two nodes is $4M$. Separating these two nodes, would increase the communication volume to $6M$ (i.e., $4M + 2M$). Since, we have already assigned node *FWD-BWD Super* to GPU to limit computation on CPU, *p16* must also be assigned to GPU.

3.4 Maximizing Memory Savings

After simplifying the data flow graph to minimize communication volume, only *g16* and *Update Super* remain to be assigned. Notice that at this point, all partitions will result in minimum communication volume, so we can prune the choices further to maximize the memory savings on GPU. Table 1 shows the memory savings of all valid partitioning strategies that minimize the communication volume. The maximum memory savings of 8x can be achieved by offloading both *g16* and *Update Super* to CPU.

Table 1: Memory savings for offload strategies that minimize communication volume compared to the baseline.

FWD-BWD	p16	g16	Update	Memory	Reduction
gpu	gpu	gpu	gpu	16M	1x (baseline)
gpu	gpu	cpu	gpu	14M	1.14x
gpu	gpu	gpu	cpu	4M	4x
gpu	gpu	cpu	cpu	4M	8x

that an offload of a model state implies that we offload the entire model state. Our analysis on the memory savings per communication volume, still holds even if we offload partial model states

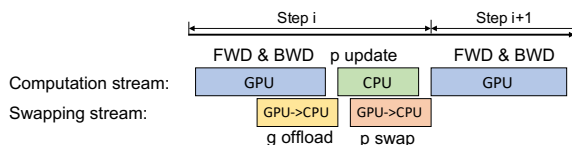


Figure 3: ZeRO-Offload training process on a single GPU.

3.5 A Unique and Optimal Offload Strategy

ZeRO-Offload allocates all the fp32 model states along with the fp16 gradients on the CPU memory, and it also computes the parameter updates on CPU. The fp16 parameters are kept on GPU and the forward and backward computations are also done on GPU.

We arrive at this offload strategy by simplifying our data flow graph and eliminating all other partitioning strategies as they do not limit CPU computation, minimize communication volume, or maximize memory savings. Therefore, ZeRO-Offload is not only optimal in terms of the aforementioned metrics, it is also unique; there can be no other strategy that can offer more memory savings than ZeRO-Offload without increasing the compute complexity on the CPU or incur additional GPU-CPU communication volume.

4 ZeRO-Offload Schedule

In this section, we discuss the concrete computation and communication schedule for implementing ZeRO-Offload on a single GPU system based on our offload strategy. We then show how we extend this schedule to work effectively on multi-GPU systems by combining our offload strategy with ZeRO data parallelism and model parallelism.

4.1 Single GPU Schedule

As discussed in Sec. 3, ZeRO-Offload partitions the data such that the fp16 parameters are stored in GPU while the fp16 gradients, and all the optimizer states such as fp32 momentum, variance and parameters are stored in CPU.

During the training, we begin by computing the loss via the forward propagation. Since the fp16 parameters are already presented on GPU, no CPU communication is required for this part of the computation. During the backward propagation on the loss, the gradient for different parameters are computed at different point in the backward schedule. ZeRO-Offload can transfer these gradients for each parameter individually or in small groups to the CPU memory immediately after they are computed. Therefore, only a small amount of memory is required to temporarily hold the gradients on the GPU memory before they are transferred to CPU memory. Furthermore, each gradient transfer can be overlapped with the backpropagation on the remainder of the backward graph, allowing ZeRO-Offload to hide a significant portion of the communication cost.

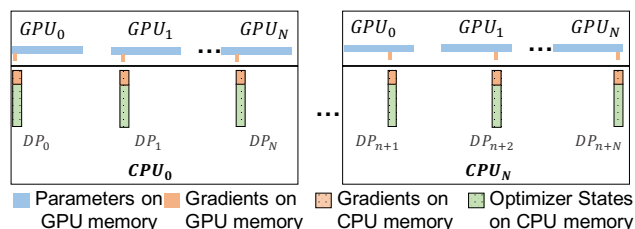


Figure 4: ZeRO-Offload data placement with multiple GPUs

After the backward propagation, ZeRO-Offload updates the fp32 parameters and the remaining optimizer states (such as momentum and variance) directly on CPU, and copies the updated fp32 parameters from the CPU memory to the fp16 parameters on the GPU memory. Figure 3 shows the computation and communication in each step of ZeRO-Offload diagrammatically, and Figure 5 shows the concrete schedule as a pseudo-code.

4.2 Scaling to Multi-GPUs

ZeRO-Offload in its entirety is a symbiotic integration of ZeRO-Offload strategy described in Sec. 3 and ZeRO-powered data parallelism discussed in Sec. 2, which allows ZeRO-Offload to scale to hundreds of GPUs efficiently. ZeRO-Offload preserves the model state partitioning strategy of ZeRO Stage-2 (optimizer state and gradient partitioning), while offloading the partitioned gradients, optimizer states and the corresponding parameter updates to CPU.

The key benefit of doing this partitioning before offloading is that for systems with more than 1 GPU, each data parallel process is only responsible for updating a subset of the parameters. The aggregated communication volume from all the data parallel GPUs to CPU remains constant, and CPU resources are used in parallel to jointly compute a single weight update. As a result, the total CPU update time decreases with increased data parallelism, since the CPU compute resources increase linearly with the increase in the number of compute nodes. This allows ZeRO-Offload to achieve very good scalability, as the overhead of communication across GPUs is offset by the reduction in the CPU optimizer step.

ZeRO-Offload partitions gradients and optimizer states among different GPUs, and each GPU offloads the partition it owns to the CPU memory and keeps it there for the entire training. During the backward propagation, gradients are computed and averaged using reduce-scatter on the GPU, and each GPU only offloads the averaged gradients belonging to its partition to the CPU memory. Once the gradients are available on the CPU, optimizer state partitions are updated in parallel by each data parallel process directly on the CPU. After the update, parameter partitions are moved back to GPU followed by an all-gather operation on the GPU similar to ZeRO-2 to gather all the parameters. Figure 4 shows the data

placement model parameters, gradients and optimizer states for ZeRO-Offload and the details of the ZeRO-Offload data parallel schedule is presented in Figure 5. The all gather operation described above is shown as a sequence of broadcast operations in the Figure.

Model Parallel training ZeRO-Offload can also work together with tensor-slicing based model parallelism (MP) frameworks such as Megatron-LM [28]. It does so by offloading the gradients, optimizer states and the optimizer computation corresponding to each MP process allowing ZeRO-Offload to train significantly larger models than possible than using model parallelism alone. Sec. 6 provides more details.

5 Optimized CPU Execution

We speedup the CPU execution time for the parameter updates with two optimizations. First, we implement a fast CPU Adam optimizer using high performance computing techniques offering significant speedup over state-of-art Pytorch implementation. Second, we develop a one-step delayed parameter update schedule that overlaps the CPU parameter update computation with the forward and backward computation on the GPU, hiding the CPU execution time when enabled.

5.1 Implementing the CPU Optimizer

We use three levels of parallelism for improving the performance of the CPU optimizer. 1) SIMD vector instruction [15] for fully exploiting the hardware parallelism supported on CPU architectures. 2) Loop unrolling [31], an effective technique for increasing instruction level parallelism that is crucial for better memory bandwidth utilization. 3) OMP multithreading for effective utilization of multiple cores and threads on the CPU in parallel. Using these technique, we present a significantly faster implementation of Adam optimizer compared to state-of-art PyTorch implementation.

Mixed Precision Training with Adam ADAM is an optimization algorithm used for deep-learning training, which takes the loss gradients together with their first and second momentums to update the parameters. Therefore, in addition to the model parameters, ADAM requires two more matrices of the same size (M) saved during the training. In the mixed precision training mode, there are two versions of the parameters stored in memory: one in fp16 (parameter16) used for computing the activations in the forward pass (on GPU), and one master copy in fp32 (parameter32) which is updated by the optimizer (on CPU). The p16 is updated with the parameter32 through *float2half* casting, at each training step. Moreover, the momentum and variance of the gradients are saved in fp32 (on CPU), to prevent the precision loss for updating the parameters. Please refer to [13] for further detail on ADAM’s algorithm.

```

1  for_parallel rank in range(world_size):
2      initialize_layers()
3      for batch in dataset:
4          x = forward(batch)
5          compute_loss(x, batch).backward()
6          backward(x.grad)
7          step()
8
9  def _is_owner(i):
10     return True if rank owns i else False
11
12  def initialize_layers():
13     for i in range(num_layers):
14         l = layers[i]
15         allocate_on_gpu l.param_fp16
16         if _is_owner(i):
17             allocate_on_cpu l.param_fp32
18             allocate_on_cpu l.optim_states_fp32
19             allocate_on_cpu l.cpu_grad
20
21  def forward(x):
22     for i in range(num_layers):
23         x = layers[i].forward(x)
24     return x
25
26  def backward(dx):
27     for i in range(num_layers, 0, -1):
28         dx=layers[i].backward(dx)
29         reduce(layers[i].grad, dest_rank
30              = _owner_rank(i))
31         if _is_owner(i) l.cpu_grad.copy(l.grad)
32         else pass
33         del layers[i].grad
34
35  def step():
36     for i in range(num_layers):
37         l=layers[i]
38         if _is_owner(i):
39             update_in_cpu(l.optim_states_fp32,
40                          l.cpu_grad,
41                          l.param_fp32)
42             l.param_fp16.copy(l.param_fp32)
43             BROADCAST(l.param_fp16, src=_owner_rank(i))

```

Figure 5: Code representing ZeRO-Offload that combines unique optimal CPU offload strategy with ZeRO-powered data parallelism.

Optimized Implementation Algorithm 1 elaborates the ADAM’s implementation detail using SIMD operations. As shown, the Adam function receives the optimizer parameters such as β_1 , β_2 , and α , and the gradient, momentum, variance and master copy of parameters (parameter32) as the input. We also use some parameters specific to the implementation, like the *simd_width* and *unroll_width*. The Adam optimizer sends back the updated variance, momentum, and parameter in both fp16 (to GPU) and fp32 (to CPU) .

We firstly read the data, including parameter, gradient, momentum and variance, into the vector registers (line 7). Then, we use several fused multiply-add (*FMA*) vector operations to preform the main execution pipeline which is repeated by the unrolling width. Note that the rest of operations, such as multiply, division, and sqrt, also run in vector mode. For the best performance we use AVX512 simd instruction set and an *unroll_width* of 8 based on auto-tuning results.

In addition to the CPU-Adam optimizer, we implement

Algorithm 2 CPU-ADAM Optimizer

Input: $p32, g32, m32, v32, \beta_1, \beta_2, \alpha, step, eps$ **Output:** $p16, p32, m32, v32$ **Parameter:** $tile_width, simd_width, unroll_width$

```
1:  $biascorrection1 \leftarrow -\alpha / (1 - \beta_1^{step})$ 
2:  $biascorrection2 \leftarrow 1 / \sqrt{1 - \beta_2^{step}}$ 
3:  $simd\_count \leftarrow sizeof(32) / simd\_width$ 
4: unroll omp parallel
5: for  $i$  in 1 to  $(simd\_count / unroll\_width)$  do
6:   ...
7:    $g_v, p_v, m_v, v_v = g32[i], p32[i], m32[i], v32[i]$ 
8:    $m_v = FMA(g_v, (1 - \beta_1), \beta_1 * m_v)$ 
9:    $v_v = FMA(g_v * g_v, (1 - \beta_2), \beta_2 * v_v)$ 
10:   $g_v = FMA(\sqrt{v_v}, biascorrection2, eps)$ 
11:   $g_v = m_m / g_v$ 
12:   $p_v = FMA(g_v, biascorrection1, p_v)$ 
13:   $p32[i], m32[i], v32[i] = p_v, m_v, v_v$ 
14:  ...
15:  IF ( $i == tile\_width$ )  $copy\_to\_gpu(p16, p32)$ 
16: end for
```

the CPU-to-GPU fp16 parameter-copy in a tiled manner (line 15). We overlap the CPU and GPU execution by parallelizing the Adam computation and copying the parameters over to GPU. As we process Adam computation of the current tile of data on CPU, we write the parameters back to GPU for the previously processed tile. This way, we reduce the idle time of GPU to start the processing of the next training step.

5.2 One-Step Delayed Parameter Update

Despite using a highly optimized CPU optimizer, the CPU computation overhead can become a bottleneck during training with very small batch sizes, when the GPU computation time is not much larger than CPU compute. For such limited cases, we develop one-step delayed parameter update (DPU) that overlaps CPU and GPU compute to hide the CPU computation overhead by delaying the parameter update by a single step. We verify that DPU does not impact the final accuracy of training in the evaluation.

DPU training schedule Figure 6 shows the workflow of ZeRO-Offload training process with delayed parameter update. ❶ The first $N - 1$ steps, are trained without DPU to avoid destabilizing the training during the early stages where gradients change rapidly. ❷ On step N , we obtain the gradients from the GPU, but we skip the CPU optimizer step, and do not update the fp16 parameters on the GPU either. ❸ At step $N + 1$, we compute the parameter updates on the CPU using gradients from step N , while computing the forward and backward pass on the GPU in parallel using parameters updated at step $N - 1$. From this step onwards, the model at $(i + 1)^{th}$ step will be trained using the parameters updated with gradients from $(i - 1)^{th}$ step instead of parameters updated at i^{th} step, overlapping CPU compute with GPU compute.

Accuracy trade-off. Since DPU changes the semantics of the training, it is reasonable to ask if there is a trade-off between model accuracy and training efficiency. To answer this

Table 2: Hardware overview of experimental system.

DGX-2 node	
GPU	16 NVIDIA Tesla V100 Tensor Core GPUs
GPU Memory	32GB HBM2 on each GPU
CPU	2 Intel Xeon Platinum 8168 Processors
CPU Memory	1.5TB 2666MHz DDR4
CPU cache	L1, L2, and L3 are 32K, 1M, and 33M, respectively
PCIe	bidirectional 32 GBps PCIe

question, we evaluated DPU on multiple training workloads and found that DPU does not hurt convergence if we introduce DPU after a few dozen iterations instead of introducing it at the beginning. Our evaluation result in Sec. 6 shows that compared with training with ZeRO-Offload only, training with delayed parameter update achieves same model training accuracy with higher training throughput.

6 Evaluation

This section seeks to answer the following questions, in comparison to the state-of-the-art:

- How does ZeRO-Offload scale the trainable model size compared to existing multi-billion parameter training solutions on a single GPU/DGX-2 node?
- What is the training throughput of ZeRO-Offload on single GPU/DGX-2 node?
- How does the throughput of ZeRO-Offload scale on up to 128 GPUs?
- What is the impact of our CPU-Adam and delay parameter update (DPU) on improving throughput, and does DPU change model convergence?

6.1 Evaluation Methodology

Testbed. For the evaluation of model scale and throughput, we conduct our experiments on a single DGX-2 node, whose details are shown in Table 2. For the evaluation of throughput scalability, we conduct experiments on 8 Nvidia DGX-2 nodes connected together with InfiniBand using a 648-port Mellanox MLNX-OS CS7500 switch.

Workloads. For the performance evaluation, we focus on evaluating GPT-2 [19] like Transformer based models [30]. We vary the hidden dimension and the number of Transformer blocks to obtain models with a different number of parameters. Note that scaling the depth alone is often not sufficient because it would make training more difficult [12]. Table 3 shows the configuration parameters used in our experiments.

For convergence analysis, such as the delayed parameter update, we use GPT-2 [19] and BERT [6], both of which are commonly used as pre-trained language models and have demonstrated superior performance in many NLP tasks (e.g., natural language understanding and inference) than recurrent neural networks or convolutional neural networks. We use BERT-large, same as the one from [6], which has 24-layer, 1024-hidden, 16-heads, and 336M parameters. Similar

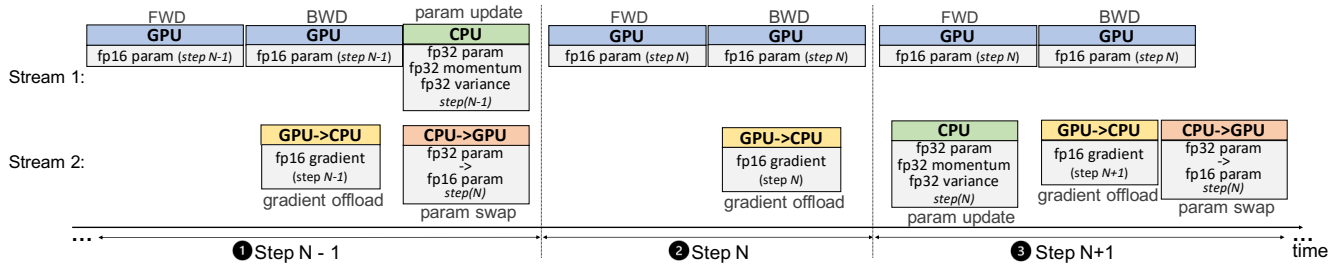


Figure 6: Delayed parameter update during the training process.

Table 3: Model configuration in evaluation.

# params	batch size per GPU	MP setting in ZeRO-Offload	# layer	hidden size
1, 2 billion	32	1	20, 40	2048
4 billion	32	1	64	2304
6, 8 billion	16	1	53, 72	3072
10, 11 billion	10, 8	1	50, 55	4096
12, 13 billion	4	1	60, 65	4096
15 billion	8	2	78	4096
20, 40, 60 billion	8	2	25, 50, 75	8192
70 billion	8	8	69	9216

to [21, 28], we fine-tune BERT on the Stanford Question Answering Dataset (SQuAD) [1], which is one of the most widely used reading comprehension benchmark [22]. Unless otherwise stated, we follow the same training procedure and hyperparameter settings as in [6, 19].

Baseline. We compare the effectiveness of ZeRO-Offload with state-of-the-art multi-billion parameter training solutions:

- PyTorch DDP: This is the existing PyTorch Transformer implementation using DistributedDataParallel [14].
- Megatron [28]: One of the current state-of-the-art multi-billion parameter model training solutions, which employs model parallelism to train up to 8.3B parameter models using 512 GPUs.
- SwapAdvisor [9]: SwapAdvisor explores a genetic algorithm to guide model-agnostic tensor swapping between GPU and CPU memory for GPU memory saving.
- L2L [18]: L2L enables training of deep Transformer networks by keeping one Transformer block at a time in GPU memory and only moves tensors in the upcoming Transformer block into GPU memory when needed.
- ZeRO-2 [21]: ZeRO extends data parallelism by eliminating memory redundancies across multiple GPUs, allowing to train models up to 170B parameters with high training throughput using 25 DGX-2 nodes. ZeRO-2 achieves the SOTA results for large model training and is a strong baseline.

6.2 Experimental Results

6.2.1 Model Scale

As an important step toward democratizing large model training, in this part, we first test the largest trainable models on a

single GPU as well as 16 GPUs in a single DGX-2 node.

Single GPU. The largest model can be trained using PyTorch DDP on a single GPU with 32GB memory is 1.4B, before running out of memory, as shown in figure 7. Both Megatron and ZeRO-2 do not increase the trainable model size on a single GPU in comparison to PyTorch, because they both utilize the aggregated GPU memory to fit larger models. In contrast, ZeRO-Offload enables 13B model training on a single GPU, which is more than 9X larger than using PyTorch, Megatron, and ZeRO-2. This is mainly because of ZeRO-Offload’s strategy for maximizing the memory savings on GPU by offloading expensive states such as optimizer states and the majority of gradients to CPU memory. The largest model can be trained with SwapAdvisor on a single GPU is 8B, which is 38% smaller than the model can be trained with ZeRO-Offload. SwapAdvisor relies on a black-box approach and uses a simulator to predict which tensors are more frequently used in order to keep them in GPU memory to maximize training throughput. The prediction can not be fully accurate, and therefore SwapAdvisor keeps more tensors in GPU memory than ZeRO-Offload does. On the other hand, L2L is able to train even larger models (e.g., 17B) on a single GPU by frequently moving weights from unused layers to CPU memory. However, the largest model size does not increase when training L2L with multiple GPUs, which is discussed next.

Multi-GPU in single DGX-2. We further perform model scale tests with 4 and 16 GPUs in a single DGX-2 node, respectively. As shown in Figure 7, the maximum trainable model size stays the same for PyTorch, L2L and SwapAdvisor, because all of them do not handle memory redundancies in data parallelism. As a result, their scalability is bounded by the model scale on a single GPU. Both Megatron and ZeRO-2 support large model training with more GPUs, but they cannot scale efficiently beyond 15B parameters, even with 16 GPUs. Megatron supports larger models than ZeRO-2, because ZeRO-2 still incurs memory redundancies on model weights. On the other hand, ZeRO-Offload easily enables training of up to 70B parameter models by partitioning and offloading optimizer states and gradients to CPU memory

combined with model parallelism. Overall, ZeRO-Offload increases the model scale on a single DGX-2 node by 50X, 4.5X, 7.8X, and 4.2X than using PyTorch, Megatron, ZeRO-2, and L2L, respectively.

6.2.2 Training Throughput

Single GPU. Next, we compare the training throughput of SwapAdvisor, L2L and ZeRO-Offload, for models with billion-scale parameters, on a single GPU. We do not include Megatron and ZeRO-2 in this comparison, because both of them cannot train models bigger than 1.4B parameters due to OOM. We evaluate SwapAdvisor, L2L and ZeRO-Offload with the same training batch size (e.g., 512) and same micro-batch sizes (shown in table 3), with gradient accumulation enabled. We also disable delayed parameter update in this experiment so that the comparison is only from the system efficiency perspective. We evaluate the performance improvement and its impact on the convergence of delayed parameter update in Section 6.2.4.

Figure 8 shows that ZeRO-Offload outperforms SwapAdvisor by 23% (up to 37%) in training throughput. SwapAdvisor relies on genetic algorithm to make tensor swapping decision, which takes hours to find an optimal tensor swapping solution in terms of maximizing the overlapping of computation and tensor swapping. Before getting the optimal tensor swapping solution, SwapAdvisor tries random tensor swapping solutions and hurts training performance.

Figure 8 shows that ZeRO-Offload outperforms L2L by 14% on average (up to 22%) in throughput (TFLOPS). The performance benefit of ZeRO-Offload comes from the following two aspects. First, ZeRO-Offload has a lower communication cost between CPU and GPU than L2L. For a model with M parameters, L2L requires $28M$ data communication volume between GPU and CPU, which is a sum of the weights, gradients, and optimizer states of each layer of the model. As analyzed in Sec. 4.1, the communication volume between CPU and GPU memory in ZeRO-Offload is $4M$, which is 7x smaller than L2L. The reduced communication volume significantly mitigates the bottleneck from CPU-GPU communication. Second, compared with L2L, the parameter update of ZeRO-Offload happens on CPU instead of GPU, but our optimized CPU-Adam implementation achieves a quite comparable parameter update performance than the PyTorch Adam implementation on GPU (evaluated in Sec. 6.2.4). Therefore, although the optimizer update on GPU in L2L is slightly faster than the optimizer update on CPU in ZeRO-Offload, the communication overhead introduced by L2L leads to an overall slower throughput than ZeRO-Offload.

Multi-GPU in single DGX-2. Next, we compare the training throughput of PyTorch, ZeRO-2, Megatron, ZeRO-Offload without model parallelism (w/o MP), and ZeRO-Offload with model parallelism (w/ MP) in one DGX-2 node. When using MP, we use a MP degree that gives the best performance

for both baseline and ZeRO-Offload. We use a total batch size of 512 for all the experiments using a combination of micro-batch per GPU and gradient accumulation. To get the best performance for each configuration, we use the largest micro batch that it can support without OOM. We exclude L2L [29] in this test because its implementation does not support multi-GPU training.

Figure 10 shows the throughput per GPU results when training on multiple GPUs. We make the following observations:

- For 1B to 15B models, ZeRO-Offload achieves the highest throughput and has up to 1.33X, 1.11X, 1.64X higher speeds than PyTorch, ZeRO-2, and Megatron, respectively. By offloading all the optimizer states to CPU with low overhead, ZeRO-Offload can train with larger micro-batch sizes giving higher throughput.
- ZeRO-2 runs out of memory once the model size is beyond 8B due to lack of enough aggregated GPU memory to store the model states on 16 GPUs. Instead, ZeRO-Offload scales to 13B, without model parallelism because it offloads optimizer states and the majority of gradients to CPU memory.
- When combined with model parallelism, ZeRO-Offload enables training up to 70B parameter models with more than 30 TFLOPS throughput per GPU. In contrast, Megatron supports only up to 15B parameter models before running out of memory, using just model parallelism.
- Compared ZeRO-Offload with ZeRO-2 and Megatron, ZeRO-Offload outperforms ZeRO-2 and Megatron in throughput for 1–8B and 1–13B parameter models, respectively. ZeRO-Offload is faster than Megatron, because it eliminates frequent communication between different GPUs and can train with larger micro batch sizes. ZeRO-Offload outperforms ZeRO-2 also due to larger micro batch sizes.

6.2.3 Throughput Scalability

We compare the throughput scalability of ZeRO-2 and ZeRO-Offload³ on up to 128 GPUs in Figure 11 and make the following key observations: First, ZeRO-Offload achieves near perfect linear speedup in terms of aggregated throughput (green line) running at over 30 TFlops per GPU (blue bars). Second, from 1 to 16 GPUs, while ZeRO-2 runs out of memory, ZeRO-Offload can effectively train the model, turning the model training from infeasible to feasible. Third, with 32 GPUs, ZeRO-Offload slightly outperforms ZeRO-2 in throughput. The improvement comes from additional memory savings on GPU from ZeRO-Offload, which allows training the model with larger batch sizes that lead to increased GPU computation efficiency. Fourth, with more GPUs (such as 64

³We do not include comparison against Megatron because it consistently performs worse than ZeRO-Offload, as shown in Figure 10. Given the communication overhead added by model parallelism, scaling out Megatron training can not achieve higher throughput than ZeRO-Offload even with linear scalability.

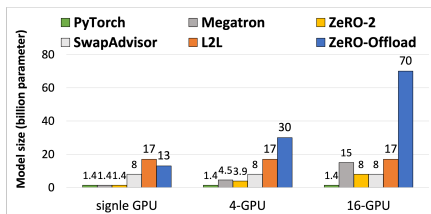


Figure 7: The size of the biggest model that can be trained on single GPU, 4 and 16 GPUs (one DGX-2 node).

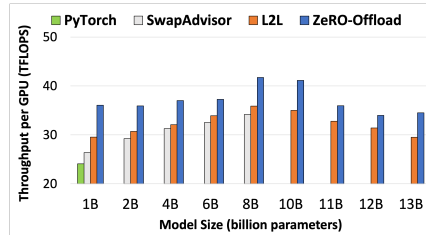


Figure 8: The training throughput with PyTorch, L2L, SwapAdvisor and ZeRO-Offload on a single GPU with a batch size of 512.

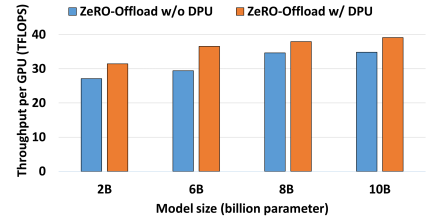


Figure 9: The training throughput is compared for w/o DPU and w/ DPU to GPT-2. Batch size is set to 8.

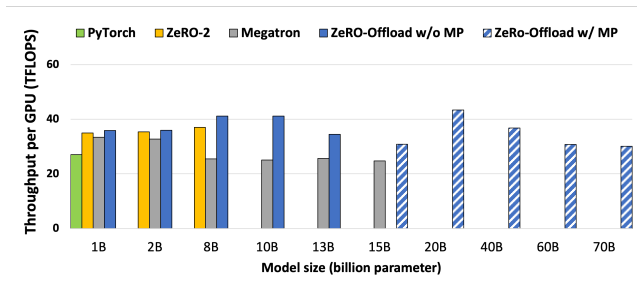


Figure 10: Training throughput with PyTorch, ZeRO-2, Megatron-LM, ZeRO-Offload without model parallelism and ZeRO-Offload with model parallelism.

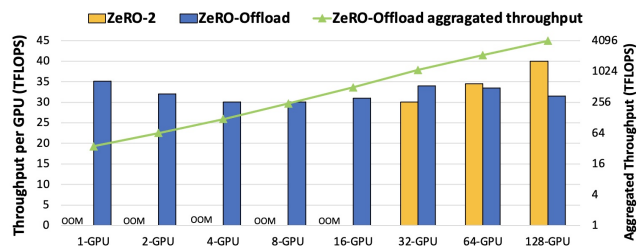


Figure 11: Comparison of training throughput between ZeRO-Offload and ZeRO-2 using 1–128 GPUs for a 10B parameter GPT2.

and 128), ZeRO-2 starts to outperform ZeRO-Offload, because both can now run similar batch sizes, achieving similar computation efficiency, whereas ZeRO-2 does not suffer from the additional overhead of CPU-GPU communication. In summary, ZeRO-Offload complements ZeRO-2 and enables large model training from a single device to thousands of devices with good computation efficiency.

6.2.4 Optimized CPU Execution

A. CPU-Adam efficiency. In this part, we evaluate our Adam implementation against the PyTorch Adam on CPU. Table 4 shows the optimizer execution time of the two implementations for model parameters from 1 to 10 billion. Compared to PyTorch (PT-CPU), CPU-Adam reduces the execution time by over 5X for all the configurations and 6.4X

for the case with 1B parameters. The CPU-Adam optimizer achieves high speedups by exploiting the instruction-level parallelism, thread-level parallelism, and the tile-based data copy scheme (as shown in line 15 of Algorithm 1). Meanwhile, although CPU-Adam has a slower speed than the PyTorch Adam implementation on GPU (PT-GPU), the performance gap is not very huge, and the CPU computation is not a bottleneck of the training throughput.

B. One-step Delayed parameter update (DPU). Figure 9 shows the comparison of the training throughput of GPT-2 with and without DPU. As shown, with DPU enabled, the training achieves 1.12–1.59, updated times higher throughput than without it, for a wide range of model sizes for a small micro batch size of 8. This is expected because DPU allows the optimizer updates to overlap with the next forward computation such that the GPU does not have to be slowed down by the CPU computation and CPU-GPU communication. But, what about accuracy?

Convergence impact We study the convergence impact of DPU on both GPT-2 and BERT. Figure 12 shows the pre-training loss curves over 100K training iterations using PyTorch (unmodified GPT-2), and Figure 13 shows the loss curves of fine-tuning Bert-large model on SQuAD using ZeRO-Offload without DPU, and ZeRO-Offload with DPU. In both cases, DPU is enabled after 40 iterations allowing the training to stabilize in its early stage before introducing DPU.

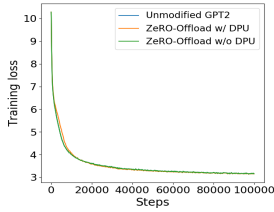
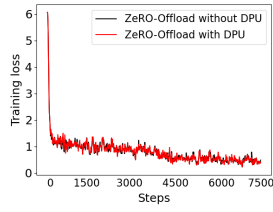
We observe that the training curves of the unmodified GPT-2 and ZeRO-Offload w/o DPU are exactly overlapped, because ZeRO-Offload w/o DPU performs only system optimizations and does not alter training dynamics. On the other hand, the training curve from ZeRO-Offload with DPU converges slightly slower at the very beginning of the training (e.g., barely can be seen at 2K-5K iterations) and quickly catches up after 5K iterations. For the remaining of the training, the training loss matches the original training until the model converges.

For Bert-Large fine-tuning, we can see that although the training losses are not exactly the same, they converge in the same trend and are largely overlapped. Without changing any hyperparameters, ZeRO-Offload + DPU achieves the same

Table 4: Adam latency (s) for PyTorch (PT) and CPU-Adam.

#Parameter	CPU-Adam	PT-CPU	PT-GPU (L2L)
1 billion	0.22	1.39	0.10
2 billion	0.51	2.75	0.26
4 billion	1.03	5.71	0.64
8 billion	2.41	11.93	0.87
10 billion	2.57	14.76	1.00

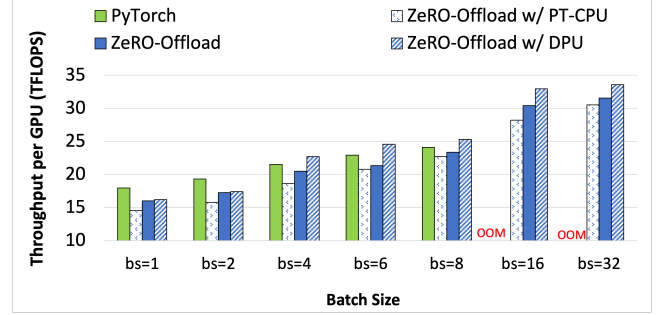
final F1 score (92.8) as the baseline. From these results on both GPT-2 pretraining, and Bert-Large fine-tuning, we empirically verify that DPU is an effective technique to improve the training throughput of ZeRO-Offload without hurting model convergence and accuracy. The 1-step staleness introduced by DPU is well tolerated by the iterative training process once the model has passed the initial training phase.

**Figure 12:** The training loss curve of unmodified GPT-2, ZeRO-Offload w/o DPU and ZeRO-Offload with DPU.**Figure 13:** The fine-tuning loss curve of BERT, ZeRO-Offload w/o DPU and ZeRO-Offload with DPU.

6.2.5 Performance Breakdown and Analysis

To better understand the performance benefit from offload strategies and optimization techniques in ZeRO-Offload, we evaluate the training throughput of PyTorch, ZeRO-Offload with PT-CPU, ZeRO-Offload with CPU-Adam (refer as ZeRO-Offload), and ZeRO-Offload with DPU. We perform the evaluation with various batch sizes with 1-billion GPT-2 model on a single GPU. Figure 14 shows the result.

From batch size 1 to 8, PyTorch outperforms ZeRO-Offload with PT-CPU by 16% on average. This is because when the model can fit on GPU memory, PyTorch does not incur any communication overhead. Meanwhile, PyTorch adopts PyTorch GPU Adam (PT-GPU) for optimizer computation on GPU. To reduce the performance loss because of communication and optimizer computation on CPU, ZeRO-Offload optimizes execution on CPU. (1) By optimizing CPU optimizer, ZeRO-Offload implements CPU-Adam and improves the performance by up to 10% compared with using offload strategy only (i.e., ZeRO-Offload with PT-CPU). (2) PyTorch outperforms ZeRO-Offload by 8% on average when the model can fit on GPU memory. As shown in table 4, the performance gap between CPU-Adam and PT-GPU is not very large. Therefore, the performance degradation from PyTorch

**Figure 14:** Comparison of training throughput with enabling offload strategies and optimization techniques step-by-step in ZeRO-Offload.

to ZeRO-Offload in Figure 14 are mainly coming from tensor migration overhead between GPU and CPU memory. (3) ZeRO-Offload further introduces one-step delayed parameter update, which overlaps computation on CPU with computation on GPU and improves performance by 7% compared with using ZeRO-Offload without DPU. In summary, leveraging optimized CPU execution, ZeRO-Offload has similar performance as PyTorch when ZeRO-Offload and PyTorch training with the same batch size on GPU.

As the batch size increases, out-of-memory on GPU memory happens in training with PyTorch. The training throughput increases in ZeRO-Offload as the batch size increasing. With unique optimal offload strategy, ZeRO-Offload outperforms PyTorch by 39% for the maximum training throughput that can be achieved on a single GPU with 1-billion model.

7 Conclusions

We presented ZeRO-Offload, a powerful GPU-CPU hybrid DL training technology with high compute efficiency and near linear throughput scalability, that can allows data scientists to train models with multi-billion parameter models even on a single GPU, without requiring any model refactoring. We open-sourced ZeRO-Offload as part of the DeepSpeed library (www.deepspeed.ai) with the hope to democratize large model training, allowing data scientist everywhere to harness the potential of truly massive DL models.

Acknowledgments

We thank the anonymous reviewers for their constructive comments. We thank our shepherd, Mark Silberstein, for his valuable feedback. This work was partially supported by U.S. National Science Foundation (CCF-1718194, CCF-1553645 and OAC-2104116) and the Chameleon Cloud.

References

- [1] The Stanford Question Answering Dataset (SQuAD) leaderboard. <https://rajpurkar.github.io/SQuAD-explorer/>.
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- [3] Yu Cao, Wei Bi, Meng Fang, and Dacheng Tao. Pre-trained language models for dialogue generation with multiple input sources, 2020.
- [4] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv: Learning*, 2016.
- [5] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’auelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, Quoc V. Le, and Andrew Y. Ng. Large scale distributed deep networks. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 25*, pages 1223–1231. Curran Associates, Inc., 2012.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [7] Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Ganger, and Phil Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training, 2018.
- [8] Mark Hildebrand, Jawad Khan, Sanjeev Trika, Jason Lowe-Power, and Venkatesh Akella. Autotm: Automatic tensor movement in heterogeneous memory systems using integer linear programming. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’20, 2020.
- [9] Chien-Chin Huang, Gu Jin, and Jinyang Li. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’20, page 1341–1355, New York, NY, USA, 2020. Association for Computing Machinery.
- [10] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism, 2018.
- [11] Hai Jin, Bo Liu, Wenbin Jiang, Yang Ma, Xuanhua Shi, Bingsheng He, and Shaofeng Zhao. Layer-centric memory reuse and data migration for extreme-scale deep learning on many-core architectures. *ACM Trans. Archit. Code Optim.*, 15(3), September 2018.
- [12] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.
- [13] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014.
- [14] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. Pytorch distributed: Experiences on accelerating data parallel training. *Proc. VLDB Endow.*, 13(12):3005–3018, 2020.
- [15] Gaurav Mitra, Beau Johnston, Alistair Rendell, Eric McCreath, and Jun Zhou. Use of simd vector operations to accelerate application code performance on low-powered arm and intel platforms. pages 1107–1116, 05 2013.
- [16] Nvidia. Automatic Mixed Precision for Deep Learning. <https://developer.nvidia.com/automatic-mixed-precision>, 2019.
- [17] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. Capuchin: Tensor-based gpu memory management for deep learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’20, page 891–905, New York, NY, USA, 2020. Association for Computing Machinery.
- [18] Bharadwaj Pudipeddi, Maral Mesmakhosroshahi, Jinwen Xi, and Sujeeth Bharadwaj. Training large neural networks with constant memory using a new execution algorithm. June 2020.
- [19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

- [20] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2020.
- [21] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.
- [22] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100, 000+ questions for machine comprehension of text. In Jian Su, Xavier Carreras, and Kevin Duh, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, pages 2383–2392. The Association for Computational Linguistics, 2016.
- [23] Jie Ren, Jiaolin Luo, Kai Wu, Minjia Zhang, Hyeran Jeon, and Dong Li. Sentinel: Efficient Tensor Migration and Allocation on Heterogeneous Memory Systems for Deep Learning. In *International Symposium on High Performance Computer Architecture (HPCA)*, 2020.
- [24] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. vdn: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *The 49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-49*, 2016.
- [25] Corby Rosset. Turing-nlg: A 17-billion-parameter language model by microsoft, 2020.
- [26] Christopher J. Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. Measuring the Effects of Data Parallelism on Neural Network Training. *Journal of Machine Learning Research*, 20, 2019.
- [27] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake A. Hechtman. Mesh-tensorflow: Deep learning for supercomputers. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 10435–10444, 2018.
- [28] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019.
- [29] Roman Tezikov. PyTorch implementation of L2L execution algorithm. <https://github.com/TezRomach/layer-to-layer-pytorch>, 2020.
- [30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008, 2017.
- [31] G. Velkoski, M. Gusev, and S. Ristov. The performance impact analysis of loop unrolling. In *2014 37th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, pages 307–312, 2014.
- [32] Oreste Villa, Mark Stephenson, David Nellans, and Stephen Keckler. Buddy Compression: Enabling Larger Memory for Deep Learning and HPC Workloads on GPUs. In *International Symposium on Computer Architecture*, 2020.
- [33] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. Superneurons: Dynamic gpu memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '18*, page 41–53, New York, NY, USA, 2018. Association for Computing Machinery.
- [34] Junzhe Zhang, Sai-Ho Yeung, Yao Shu, Bingsheng He, and Wei Wang. Efficient memory management for gpu-based deep learning systems. *CoRR*, abs/1903.06631, 2019.
- [35] M. A. Zinkevich, M. Weimer, A. Smola, and L. Li. Parallelized Stochastic Gradient Descent. In *International Conference on Neural Information Processing Systems*, 2010.